



This document outlines a technical research report, not the research report itself, and an implementation plan for the all new Neural Net architecture, **CondorNet™**—a fintech-specialized hybrid neural architecture designed for **SPY options trading**. CondorNet combines the strengths of three fundamentally different model families:

- **Neural CDE** for continuous-time, physics-consistent dynamics
- **Mamba2 (SSM/SSD)** for high-throughput long-context sequence modeling
- **TFT-style mechanisms** for interpretable feature selection and temporal attention

Crucially, it is built to natively support my latest **predicate/inequality reasoning system**, including recursive “set-of-sets” (super-set) composition—because in real markets, the most valuable signals are often *structured*, *rare*, and *relational*, not just statistically smooth.

In the pages that follow, I do three things:

1. Define a shared mathematical interface that makes CDE, Mamba2, and TFT **comparable and composable**—this is the key to identifying “like terms.”
2. Propose a CondorNet architecture that is **not** a naive “stack everything” approach, but a disciplined fusion: operator-splitting + mixture-of-experts routing + stability constraints + predicate-driven decision logic.
3. Provide drop-in Python modules (backbone, training skeleton, stability utilities) designed to snap into the existing CondorBrain pipeline, including predicate discovery and recursive evaluation.

It’s worth emphasizing: **Mamba2 is powerful**. But like any standalone architecture, it has regions where it shines and regions where it struggles—especially under the kinds of extreme, discontinuous, regime-switching behaviors that show up in options data. After evaluating Neural CDE, Mamba2, and TFT side-by-side, I reached a clear conclusion: each model family contains

high-value components, and each contains components that can become liabilities if used uncritically.

That realization leads to the central question behind this work:

What if we stop choosing one model—and instead extract only the parts we need from each, while actively blocking the failure modes?

Answering that question forced an uncomfortable but productive kind of work: repeatedly translating between very different mathematical languages (continuous-time dynamics, discrete state-space scans, and attention/selection mechanisms), and identifying the points where terms can be aligned, substituted, or reduced into a stable, optimizable whole. It took a lot of iteration—backtesting, debugging, writing, and verifying—because the goal is not “a clever model,” as that goes without saying, but a model that is **auditable, reproducible, sustainable, and scalable**.

By background, I’ve spent decades building high-performance systems: digital signal processing, GPU/CUDA optimization, and automated trading research and engineering—including work for larger institutions. I’ve also run software technology companies for many years, filed patents, and learned the hard way that execution matters: documentation, traceability, and the ability to hand work off cleanly are not luxuries; they’re survival traits in serious engineering. So, if you are wonder why I am constantly writing papers while models are training, or backtesting, or some other time consuming task in the background – it is far easier to thoroughly document everything being worked on when its fresh in our minds, then try and go back and document it even a couple days later. This has always been my *modus operandi*, especially in Computational Physics and Software Engineering.

One modern advantage I didn’t have in earlier years is the rise of advanced AI engines. Used correctly, they function like a fast, always-available engineering team—especially when paired with real code, diagrams, and rigorous specs. They don’t replace judgment or domain expertise, but they dramatically compress iteration time when the prompts and artifacts are precise.

Finally, a quick note on a result that initially looked “obvious”: in early comparisons, a Neural CDE model appeared to outperform a Mamba2 model because one remained numerically stable while the other collapsed to NaNs. But a deeper read suggested something more important: the “failed” model wasn’t useless—it was revealing **which components are essential, and which must be constrained or re-engineered**. That is the *real purpose* of CondorNet: not stacking models, but designing a *custom architecture* aligned to the realities of options trading—fast, regime-aware, predicate-driven, stability-controlled, and engineered for clean integration into a broader risk and execution system.

If this approach works the way the mathematics suggests, it won’t be limited to SPY, and certainly not limited to fintech. Why start with SPY options—because of liquidity, structure, and available data—the right proving ground.

0) Why Mamba2 “blew up” while the Neural CDE held

From the comparison JSON, the Mamba2 model’s mean outputs are all **NaN**, while the Neural CDE outputs remain finite:

- Neural CDE `output_mean` is finite across all 10 heads.
- Mamba2 `output_mean` is `[nan, ..., nan]` (implosion/explosion → numerical instability).

This pattern is extremely consistent with **state-space scan numeric instability** (SSD kernels + BF16 + large activation magnitudes), *especially when predicate-augmented inputs create occasional extreme values or very sharp discontinuities*.

Key reason: **Mamba2’s internal recurrence is multiplicative in a way CDEs often aren’t**.

1) A unifying mathematical “interface” across CDE, Mamba2, TFT

To fuse architectures without “duct taping,” we need a shared abstraction.

1.1 The common object: a latent controlled dynamical system

We define a latent state $z(t) \in \mathbb{R}^d$ driven by a control path $X(t) \in \mathbb{R}^F$ (the 54 base features + predicate channels):

$$X(t) = [x_1(t), \dots, x_F(t)], z(t) \in \mathbb{R}^d$$

- $X(t)$: interpolated multi-feature path from the windowed data.
- $z(t)$: latent state used by output heads (entry/exit/ROI/etc.).

Now each backbone is just a different discretization/parameterization of the same idea:

1.2 Neural CDE = continuous-time controlled evolution

Neural CDE form:

$$z(t_1) = z(t_0) + \int_{t_0}^{t_1} f_{\theta}(z(t)) dX(t)$$

- $f_{\theta}: \mathbb{R}^d \rightarrow \mathbb{R}^{d \times F}$
- The integral is a controlled integral driven by the path $X(t)$.

Why it stays stable in practice:

If f_θ is Lipschitz and we normalize/clamp inputs, the solution tends to remain bounded; CDE implementations often include norm layers and smooth interpolation.

1.3 Mamba2 (SSM/SSD) = discrete state-space scan

Canonical (linear) SSM discretization:

$$s_{t+1} = A s_t + B u_t, y_t = C s_t$$

Mamba2 uses a *structured*, input-dependent, gated parameterization and fast scan kernels, but it still behaves like:

$$s_{t+1} = \Phi(u_t) s_t + \Psi(u_t)$$

- $\Phi(u_t)$ must be **contractive** (spectral radius < 1) to avoid blow-ups.
- In BF16, if Φ approaches 1 or slightly exceeds 1, long scans can explode; or if there's internal exponentiation / $\exp 2$, it can overflow.

Interpretability implication:

Attribution methods on Mamba2 must often operate on (a) input→state impulse response, (b) gate activations, and (c) stability measures (state growth/decay).

1.4 TFT = interpretable variable selection + attention on time

TFT's most valuable "fintech" contributions are:

1. **Variable Selection Network (VSN)**: produces sparse, interpretable weights over features.
2. **Temporal attention**: focuses on salient time steps.
3. **Gated residual networks**: stabilize learning and separate signal from noise.

A simplified VSN for time-varying inputs:

$$\alpha_t = \text{softmax}(W_2 \sigma(W_1 x_t)) \in \mathbb{R}^F$$
$$\tilde{x}_t = \sum_{j=1}^F \alpha_{t,j} V_j(x_{t,j})$$

- $\alpha_{t,j}$: interpretable feature importance at time t .
- V_j : per-feature embedding/projection.

2) CondorNet: the fusion that “snaps into place” mathematically

2.1 Operator-splitting fusion (physics-style)

Instead of stacking CDE→Mamba→TFT (which is unstable and memory heavy), we treat them as **three operators** that each produce a candidate latent update:

- $\mathcal{F}_{\text{CDE}}$: smooth continuous-time dynamics
- $\mathcal{F}_{\text{SSM}}$: long-context discrete scan
- $\mathcal{F}_{\text{TFT}}$: sparse interpretable selection + attention

At each time step/window, produce three latent summaries:

$$h^{\text{cde}} \in \mathbb{R}^d, h^{\text{ssm}} \in \mathbb{R}^d, h^{\text{tft}} \in \mathbb{R}^d$$

Then fuse them with a **regime-aware simplex gate** $g \in \Delta^3$:

$$\begin{aligned} g &= \text{softmax}(G_\phi(\rho)), g \in \mathbb{R}^3, \sum_i g_i = 1, g_i \geq 0 \\ h &= g_1 h^{\text{cde}} + g_2 h^{\text{ssm}} + g_3 h^{\text{tft}} \end{aligned}$$

Where ρ is a compact **regime descriptor** (IVR/ADX/MTF/gap risk/fuzzy gates/predicate density).

This does **three** crucial things:

1. We get **apples-to-apples outputs** (same latent size, same heads).
2. We avoid “Mamba meltdown everywhere” by letting the gate **down-weight** the SSM branch when it is unstable.
3. We can audit interpretability in a *consistent basis*: compare $(g, h^{\text{cde}}, h^{\text{ssm}}, h^{\text{tft}})$.

2.2 Stability constraint: contractive SSM branch

To stop Mamba2 collapse, enforce a contractive prior.

Define per-step state growth:

$$\gamma_t = \frac{\|s_{t+1}\|_2}{\|s_t\|_2 + \varepsilon}$$

Add a penalty that pushes $\gamma_t \leq 1$:

$$\mathcal{L}_{\text{stab}} = \frac{1}{T} \sum_{t=1}^{T-1} \max(0, \gamma_t - 1)^2$$

- This is a **physics metric** (growth rate) turned into a regularizer.
- It aligns directly with my latest “stability metrics” idea in the auditor (“Lipschitz stability”). The same metric can be computed online in the audit pipeline.

Practical T4 trick: compute γ_t using **FP32 only** on a small subset of batches; do not backprop through the full time scan every step.

2.3 Predicate/Inequality logic as a sparse event process

The new predicate engine defines an enormous inequality search space and uses differentiable sparse selection to learn which are predictive.

predicate_discovery

We should treat predicate activations as **events** $p_k(t) \in [0,1]$ (or logit scores), not just extra dense inputs.

Let $p(t) \in \mathbb{R}^K$ be the vector of top-K predicate activations ($K \leq 256$ active from 2048 slots), with possible recursive composition.

recursive_evaluator

Define a “predicate intensity”:

$$\lambda(t) = \sum_{k=1}^K w_k p_k(t)$$

Use $\lambda(t)$ to:

1. **Modulate the fusion gate** (if predicate density spikes, change branch weight):

$$g = \text{softmax}(G_\phi([\rho, \lambda]))$$

2. **Create a fast “bloom-like” lookup** of predicate signatures for routing.

2.4 Bloom-filter analogy that is actually useful here

A classic Bloom filter is non-differentiable, but we can implement a **learned differentiable hashing gate**:

1. Convert predicate IDs to embeddings $e_k \in \mathbb{R}^d$.
2. Project into m “bit-channels” with sign / sigmoid:

$$b_k = \sigma(W e_k) \in (0,1)^m$$
$$B(t) = 1 - \prod_{k: p_k(t) \text{ active}} (1 - b_k)$$

- $B(t) \in (0,1)^m$ acts like a **soft Bloom signature**.
- m can be small (e.g. 256–1024), giving us a fast vector that summarizes which inequality families are firing.
- This signature routes the model: gate, MoE, or “risk override” head.

This is the “ $k=15$ ” intuition, but made differentiable and stable.

3) Apples-to-apples comparison metrics for CondorNet (and for CDE vs Mamba)

Because CondorNet produces **branch-separable** outputs, we can compare both:

- Whole model outputs (like we do now)
- Branch outputs and gate behaviors (new and very diagnostic)

3.1 Branch-normalized attribution (fixes unfair comparisons)

Permutation importance on raw outputs is unfair if one model is “sharper” or has higher variance.

Define head j prediction function f_j , and normalize by each model’s output scale:

$$\tilde{f}_j(x) = \frac{f_j(x) - \mu_j}{\sigma_j + \varepsilon}$$

Then compute permutation importance on $\tilde{f}$, not f . This removes a major “apples vs oranges” artifact.

3.2 Inequality reasoning alignment across architectures

The core issue: a CDE can internalize inequality patterns as smooth state trajectories; Mamba can internalize them as scan-based gating patterns. We need a shared representation.

Define a **Predicate Reasoning Vector** (PRV) computed outside the backbone:

$$r = \text{Pool}_t(\psi(p(t))) \in \mathbb{R}^{d_r}$$

Where ψ is a small MLP, and Pool can be mean/max/attention pooling.

Then each architecture gets the same additional conditioning:

$$h \leftarrow h + Ur$$

Now the audit can compare r directly across architectures (distribution shift, MI with outputs, etc.) without peeking into model internals.

4) Memory and numeric stability on a T4 (what actually matters)

A Tesla T4 has ~16GB VRAM; the failure we saw (“Killed”) also suggests **host RAM pressure** from pandas/CSV or big intermediate arrays.

4.1 Data streaming (host RAM fix)

The new auditor already does chunk streaming. Good. Keep these rules:

- `pd.read_csv(..., chunksize=100_000)` only (never load full 5.8GB).
- Maintain a fixed circular buffer of size `seq_len`.
- Use reservoir sampling, not list append.

(The “Ultra-Fast GPU Dataset” trainer emphasizes avoiding transfers; the auditor should do the opposite: avoid GPU residency except for batch inference.)

`train_condor_brain`

4.2 GPU OOM fix: never store activation tensors for entire sample set

Interpretability often OOMs because scripts collect:

- per-sample gradients
- per-sample intermediate activations
- large Jacobians / Fisher blocks
- SHAP approximations with many forward passes

Rule: compute metrics as **streaming accumulators**.

Example (gradient saliency) accumulator:

$$S = \frac{1}{N} \sum_{i=1}^N |\nabla_x f(x_i)|$$

Compute per-batch, add to `s_sum`, divide once at end. Do **not** store per-sample gradients.

4.3 Mamba2 numeric stability: mixed precision policy (critical)

For Mamba2 branch:

- Keep Mamba core in **FP32** even if the rest runs BF16.
- Run `autocast` OFF inside the Mamba call, or cast inputs to FP32 and ensure outputs are FP32.

This alone prevents a large class of NaN explosions in SSD kernels.

4.4 Hessian / Fisher: use Hutchinson + diagonal Fisher

Full Hessian eigenspectrum is not T4-friendly.

Instead:

Diagonal Fisher (cheap, stable):

$$F_{\text{diag}} \approx \mathbb{E} [(\nabla_{\theta} \ell)^2]$$

Compute $(\nabla_{\theta} \ell)$ on small batches; square; accumulate; done.

Top eigenvalues via Hutchinson (optional):

$$\text{Tr}(H) \approx \frac{1}{M} \sum_{m=1}^M v_m^\top H v_m, v_m \sim \{\pm 1\}^d$$

But implement only if needed; otherwise “physics stability + fisher diag + saliency” already gives us 90% of what we want.

5) CondorNet prototype modules (drop-in Python)

Below are **new scripts/modules** intended to sit next to the current `condor_brain.py` and reuse the existing conventions (input_dim=54, seq_len=240, 10 heads, predicate slots, etc.).

5.1 *intelligence/condor_net.py* — Hybrid backbone (CDE + Mamba2 + TFT) with stable fusion

```
1. # intelligence/condor_net.py
2. import math
3. from dataclasses import dataclass
4. from typing import Dict, Optional, Tuple
5.
6. import torch
7. import torch.nn as nn
8. import torch.nn.functional as F
9.
10. try:
11.     from mamba_ssm import Mamba
12.     HAS_MAMBA = True
13. except Exception:
14.     HAS_MAMBA = False
15.
16. try:
17.     # The repo already references this in condor_brain.py
18.     from intelligence.models.neural_cde import NeuralCDE
19.     HAS_CDE = True
20. except Exception:
21.     HAS_CDE = False
22.
23.
24. # -----
25. # TFT-style building blocks
26. # -----
27.
28. class GRN(nn.Module):
29.     """Gated Residual Network (TFT-style) for stable nonlinear transforms."""
30.     def __init__(self, d_in: int, d_hidden: int, d_out: int, dropout: float = 0.1):
31.         super().__init__()
32.         self.fc1 = nn.Linear(d_in, d_hidden)
33.         self.fc2 = nn.Linear(d_hidden, d_out)
34.         self.gate = nn.Linear(d_in, d_out)
35.         self.ln = nn.LayerNorm(d_out)
36.         self.dropout = nn.Dropout(dropout)
37.
38.     def forward(self, x: torch.Tensor) -> torch.Tensor:
39.         h = F.elu(self.fc1(x))
40.         h = self.dropout(self.fc2(h))
```

```
41.         g = torch.sigmoid(self.gate(x))
42.         y = g * h + (1.0 - g) * x[..., : h.shape[-1]]
43.         return self.ln(y)
44.
45.
46. class VariableSelection(nn.Module):
47.     """
48.     TFT-like variable selection:
49.     Produces per-time feature weights alpha_t and a projected feature embedding.
50.     """
51.     def __init__(self, n_features: int, d_model: int, d_hidden: int = 128, dropout: float =
0.1):
52.         super().__init__()
53.         self.n_features = n_features
54.         self.score = GRN(n_features, d_hidden, n_features, dropout=dropout)
55.         self.value = nn.Linear(n_features, d_model)
56.
57.     def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
58.         # x: (B, T, F)
59.         logits = self.score(x) # (B, T, F)
60.         alpha = torch.softmax(logits, dim=-1) # (B, T, F)
61.         x_weighted = x * alpha # (B, T, F)
62.         emb = self.value(x_weighted) # (B, T, d_model)
63.         return emb, alpha
64.
65.
66. class TemporalSelfAttention(nn.Module):
67.     """Lightweight temporal attention (interpretable-ish)."""
68.     def __init__(self, d_model: int, n_heads: int = 4, dropout: float = 0.1):
69.         super().__init__()
70.         self.attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
71.         self.ln = nn.LayerNorm(d_model)
72.
73.     def forward(self, h: torch.Tensor) -> torch.Tensor:
74.         # h: (B, T, d)
75.         a, _ = self.attn(h, h, h, need_weights=False)
76.         return self.ln(h + a)
77.
78.
79. # -----
80. # Predicate routing helpers
81. # -----
82.
83. class SoftBloomSignature(nn.Module):
84.     """
85.     Differentiable 'Bloom-like' signature for active predicates.
86.     Input: predicate activations p(t) in (B,T,K)
87.     Output: signature B(t) in (B,T,M)
88.     """
89.     def __init__(self, k_pred: int, m_bits: int = 256, d_hidden: int = 128):
90.         super().__init__()
91.         self.proj = nn.Sequential(
92.             nn.Linear(k_pred, d_hidden),
93.             nn.GELU(),
94.             nn.Linear(d_hidden, m_bits),
95.         )
96.
97.     def forward(self, p: torch.Tensor) -> torch.Tensor:
98.         # p: (B,T,K)
99.         return torch.sigmoid(self.proj(p))
100.
101.
102. # -----
103. # CondorNet Backbone
104. # -----
```

```
105.
106. @dataclass
107. class CondorNetConfig:
108.     input_dim: int = 54
109.     d_model: int = 256
110.     n_layers: int = 6
111.     dropout: float = 0.1
112.     tft_heads: int = 4
113.     mamba_layers: int = 2
114.     mamba_d_state: int = 16
115.     # predicate routing
116.     pred_dim: int = 0          # set >0 if we provide predicate channels
117.     bloom_bits: int = 256
118.     # numeric policy
119.     force_mamba_fp32: bool = True
120.
121.
122. class CondorNetBackbone(nn.Module):
123.     """
124.     Hybrid backbone:
125.     - CDE branch: smooth continuous-time dynamics (if available)
126.     - Mamba branch: long-context scan (FP32 optional)
127.     - TFT branch: variable selection + temporal attention
128.     Fusion:
129.     h = sum_i g_i * h_i, with g = softmax(G([regime, predicate_signature]))
130.     """
131.     def __init__(self, cfg: CondorNetConfig):
132.         super().__init__()
133.         self.cfg = cfg
134.
135.         # Shared input projection for gating/regime inference
136.         self.in_proj = nn.Sequential(
137.             nn.Linear(cfg.input_dim + cfg.pred_dim, cfg.d_model),
138.             nn.LayerNorm(cfg.d_model),
139.             nn.GELU(),
140.             nn.Dropout(cfg.dropout),
141.         )
142.
143.         # ----- TFT branch -----
144.         self.vsn = VariableSelection(cfg.input_dim + cfg.pred_dim, cfg.d_model,
145. dropout=cfg.dropout)
146.         self.tft_attn = TemporalSelfAttention(cfg.d_model, n_heads=cfg.tft_heads,
147 dropout=cfg.dropout)
148.         self.tft_ff = nn.Sequential(
149.             nn.Linear(cfg.d_model, 4 * cfg.d_model),
150.             nn.GELU(),
151.             nn.Dropout(cfg.dropout),
152.             nn.Linear(4 * cfg.d_model, cfg.d_model),
153.             nn.LayerNorm(cfg.d_model),
154.         )
155.
156.         # ----- Mamba branch -----
157.         self.has_mamba = HAS_MAMBA
158.         if self.has_mamba:
159.             self.mamba = nn.ModuleList([
160.                 Mamba(
161.                     d_model=cfg.d_model,
162.                     d_state=cfg.mamba_d_state,
163.                     d_conv=4,
164.                     expand=2,
165.                 ) for _ in range(cfg.mamba_layers)
166.             ])
167.             self.mamba_ln = nn.LayerNorm(cfg.d_model)
168.
169.         # ----- CDE branch -----
```

```
168.         self.has_cde = HAS_CDE
169.         if self.has_cde:
170.             # Assume NeuralCDE takes (B,T,F) -> (B,T,d_model) or similar
171.             self.cde = NeuralCDE(
172.                 input_dim=cfg.input_dim + cfg.pred_dim,
173.                 hidden_dim=cfg.d_model,
174.                 num_layers=cfg.n_layers,
175.             )
176.             self.cde_ln = nn.LayerNorm(cfg.d_model)
177.
178.         # Predicate signature (optional)
179.         if cfg.pred_dim > 0:
180.             self.bloom = SoftBloomSignature(cfg.pred_dim, m_bits=cfg.bloom_bits)
181.             gate_in = cfg.d_model + cfg.bloom_bits
182.         else:
183.             self.bloom = None
184.             gate_in = cfg.d_model
185.
186.         # Fusion gate: outputs 3 weights (cde, mamba, tft)
187.         self.gate = nn.Sequential(
188.             nn.Linear(gate_in, cfg.d_model),
189.             nn.GELU(),
190.             nn.Linear(cfg.d_model, 3),
191.         )
192.
193.     def forward(
194.         self,
195.         x: torch.Tensor,                # (B,T,F)
196.         p: Optional[torch.Tensor] = None # (B,T,K_pred) if used
197.     ) -> Dict[str, torch.Tensor]:
198.         if p is not None:
199.             x_in = torch.cat([x, p], dim=-1)
200.         else:
201.             x_in = x
202.
203.         # Shared summary for gating: use last token of in_proj
204.         h0 = self.in_proj(x_in)          # (B,T,d)
205.         h_summary = h0[:, -1, :]         # (B,d)
206.
207.         # Gate conditioning with predicate signature if present
208.         if self.bloom is not None and p is not None:
209.             sig = self.bloom(p)           # (B,T,M)
210.             sig_summary = sig[:, -1, :]   # (B,M)
211.             gate_in = torch.cat([h_summary, sig_summary], dim=-1)
212.         else:
213.             sig = None
214.             gate_in = h_summary
215.
216.         g_logits = self.gate(gate_in)     # (B,3)
217.         g = torch.softmax(g_logits, dim=-1) # (B,3)
218.
219.         # -----
220.         # TFT branch
221.         # -----
222.         ht, alpha = self.vsn(x_in)        # (B,T,d), (B,T,F)
223.         ht = self.tft_attn(ht)            # (B,T,d)
224.         ht = self.tft_ff(ht)              # (B,T,d)
225.         ht_sum = ht[:, -1, :]             # (B,d)
226.
227.         # -----
228.         # Mamba branch
229.         # -----
230.         if self.has_mamba:
231.             hm = h0
232.             if self.cfg.force_mamba_fp32:
```

```
233.         hm = hm.float()
234.         for layer in self.mamba:
235.             hm = layer(hm)
236.             hm = self.mamba_ln(hm)
237.             hm_sum = hm[:, -1, :]
238.         else:
239.             hm = torch.zeros_like(h0)
240.             hm_sum = hm[:, -1, :]
241.
242.         # -----
243.         # CDE branch
244.         # -----
245.         if self.has_cde:
246.             hc = self.cde(x_in)                # expect (B,T,d)
247.             hc = self.cde_ln(hc)
248.             hc_sum = hc[:, -1, :]
249.         else:
250.             hc = torch.zeros_like(h0)
251.             hc_sum = hc[:, -1, :]
252.
253.         # -----
254.         # Fusion
255.         # -----
256.         # expand weights for time fusion
257.         g_c, g_m, g_t = g[:, 0:1], g[:, 1:2], g[:, 2:3] # (B,1)
258.         h_fused = (g_c.unsqueeze(1) * hc) + (g_m.unsqueeze(1) * hm) + (g_t.unsqueeze(1) * ht)
259.         h_fused_sum = h_fused[:, -1, :]
260.
261.         return {
262.             "h_seq": h_fused,                # (B,T,d)
263.             "h": h_fused_sum,                # (B,d)
264.             "g": g,                          # (B,3)
265.             "alpha": alpha,                  # (B,T,F_total)
266.             "sig": sig if sig is not None else torch.empty(0, device=x.device),
267.             "h_cde": hc_sum,
268.             "h_mamba": hm_sum,
269.             "h_tft": ht_sum,
270.         }
```

What this gives us immediately:

- A **stable fusion gate** that can *learn* when Mamba is unsafe.
- TFT-style **feature weights** (alpha) for interpretability.
- Optional predicate signature routing.

5.2 *intelligence/condor_net_heads.py* — heads + stability penalty hooks

```
1. # intelligence/condor_net_heads.py
2. from dataclasses import dataclass
3. from typing import Dict
4. import torch
5. import torch.nn as nn
6. import torch.nn.functional as F
7.
8. @dataclass
9. class HeadConfig:
10.     d_model: int = 256
11.     n_outputs: int = 10
12.     dropout: float = 0.1
13.
```

```
14. class CondorHeads(nn.Module):
15.     """
16.     Same 10-head ordering we already use:
17.     [call_offset, put_offset, wing_width, dte, pop, roi, max_loss, confidence, entry, exit]
18.     """
19.     def __init__(self, cfg: HeadConfig):
20.         super().__init__()
21.         self.net = nn.Sequential(
22.             nn.Linear(cfg.d_model, 2 * cfg.d_model),
23.             nn.GELU(),
24.             nn.Dropout(cfg.dropout),
25.             nn.LayerNorm(2 * cfg.d_model),
26.             nn.Linear(2 * cfg.d_model, cfg.n_outputs),
27.         )
28.
29.     def forward(self, h: torch.Tensor) -> torch.Tensor:
30.         return self.net(h)
31.
32. def stability_growth_penalty(states: torch.Tensor, eps: float = 1e-6) -> torch.Tensor:
33.     """
34.     Growth penalty for a state sequence s_t (B,T,d):
35.     gamma_t = ||s_{t+1}|| / (||s_t|| + eps)
36.     L = mean( max(0, gamma_t - 1)^2 )
37.     """
38.     s = states
39.     n0 = torch.linalg.norm(s[:, :-1, :], dim=-1) + eps
40.     n1 = torch.linalg.norm(s[:, 1:, :], dim=-1) + eps
41.     gamma = n1 / n0
42.     return torch.mean(torch.clamp(gamma - 1.0, min=0.0) ** 2)
```

5.3 intelligence/train_condor_net.py—trainer skeleton (T4-safe)

This is intentionally designed to mirror the training scripts: robust normalization, BF16, and safe clamping; but with **Mamba-FP32 branch** and optional predicate channels.

```
1. # intelligence/train_condor_net.py
2. import os
3. os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
4. os.environ.setdefault("CUDA_MODULE_LOADING", "LAZY")
5.
6. import argparse
7. import numpy as np
8. import pandas as pd
9. import torch
10. from torch.utils.data import IterableDataset, DataLoader
11. from torch.amp import autocast, GradScaler
12. from tqdm import tqdm
13.
14. from intelligence.canonical_feature_registry import FEATURE_COLS_V22
15. from intelligence.condor_net import CondorNetConfig, CondorNetBackbone
16. from intelligence.condor_net_heads import HeadConfig, CondorHeads, stability_growth_penalty
17.
18. # -----
19. # Minimal streaming dataset
20. # -----
21.
22. class WindowStream(IterableDataset):
23.     def __init__(self, csv_path, seq_len, n_samples, seed=42, chunksize=100_000):
24.         self.csv_path = csv_path
25.         self.seq_len = seq_len
26.         self.n_samples = n_samples
```

```
27.         self.seed = seed
28.         self.chunksize = chunksize
29.
30.     def __iter__(self):
31.         rng = np.random.default_rng(self.seed)
32.         buf = np.zeros((self.seq_len, len(FEATURE_COLS_V22)), dtype=np.float32)
33.         buf_count = 0
34.
35.         # Reservoir of windows
36.         reservoir = []
37.         seen = 0
38.
39.         usecols = FEATURE_COLS_V22
40.         dtypes = {c: np.float32 for c in usecols}
41.
42.         for chunk in pd.read_csv(self.csv_path, usecols=usecols, dtype=dtypes,
chunksize=self.chunksize):
43.             Xc = chunk.to_numpy(copy=False)
44.             for i in range(Xc.shape[0]):
45.                 buf[buf_count % self.seq_len] = np.nan_to_num(Xc[i], nan=0.0, posinf=0.0,
neginf=0.0)
46.                 buf_count += 1
47.                 if buf_count < self.seq_len:
48.                     continue
49.
50.                 # build window
51.                 start = buf_count % self.seq_len
52.                 window = np.concatenate([buf[start:], buf[:start]], axis=0) # (T,F)
53.
54.                 seen += 1
55.                 if len(reservoir) < self.n_samples:
56.                     reservoir.append(window)
57.                 else:
58.                     j = rng.integers(0, seen)
59.                     if j < self.n_samples:
60.                         reservoir[j] = window
61.
62.             rng.shuffle(reservoir)
63.             for w in reservoir:
64.                 yield w
65.
66. def main():
67.     ap = argparse.ArgumentParser()
68.     ap.add_argument("--data", required=True)
69.     ap.add_argument("--seq-len", type=int, default=240)
70.     ap.add_argument("--samples", type=int, default=2000)
71.     ap.add_argument("--batch-size", type=int, default=32)
72.     ap.add_argument("--epochs", type=int, default=5)
73.     ap.add_argument("--lr", type=float, default=1e-4)
74.     ap.add_argument("--out", default="models/condor_net.pth")
75.     args = ap.parse_args()
76.
77.     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
78.
79.     ds = WindowStream(args.data, args.seq_len, args.samples)
80.     dl = DataLoader(ds, batch_size=args.batch_size, num_workers=0)
81.
82.     cfg = CondorNetConfig(input_dim=len(FEATURE_COLS_V22), d_model=256, pred_dim=0,
force_mamba_fp32=True)
83.     backbone = CondorNetBackbone(cfg).to(device)
84.     heads = CondorHeads(HeadConfig(d_model=cfg.d_model, n_outputs=10)).to(device)
85.
86.     opt = torch.optim.AdamW(list(backbone.parameters()) + list(heads.parameters()), lr=args.lr,
weight_decay=1e-2)
87.     scaler = GradScaler(enabled=(device.type == "cuda"))
```

```
88.
89.     backbone.train()
90.     heads.train()
91.
92.     for ep in range(args.epochs):
93.         pbar = tqdm(dl, desc=f"Epoch {ep+1}/{args.epochs}")
94.         for xb in pbar:
95.             xb = xb.to(device, non_blocking=True) # (B,T,F)
96.
97.             # TODO: actual targets go here (y)
98.             # For smoke tests, use a dummy objective that keeps numerics stable.
99.             # Replace with CondorLoss / CompositeCondorLoss from the pipeline.
100.            with autocast(device_type="cuda", enabled=(device.type=="cuda"),
dtype=torch.bfloat16):
101.                out = backbone(xb)
102.                yhat = heads(out["h"]) # (B,10)
103.
104.                # dummy target: encourage finite outputs + small magnitude
105.                loss_main = torch.mean(yhat * yhat)
106.
107.                # stability penalty primarily on Mamba sequence if present
108.                loss_stab = stability_growth_penalty(out["h_seq"].float())
109.
110.                loss = loss_main + 0.1 * loss_stab
111.
112.                opt.zero_grad(set_to_none=True)
113.                scaler.scale(loss).backward()
114.                torch.nn.utils.clip_grad_norm_(list(backbone.parameters()) +
list(heads.parameters()), 1.0)
115.                scaler.step(opt)
116.                scaler.update()
117.
118.                pbar.set_postfix(loss=float(loss.detach().cpu()))
119.
120.            ckpt = {
121.                "backbone": backbone.state_dict(),
122.                "heads": heads.state_dict(),
123.                "cfg": cfg.__dict__,
124.            }
125.            torch.save(ckpt, args.out)
126.            print(f"[OK] Saved: {args.out}")
127.
128. if __name__ == "__main__":
129.     main()
```

Why the dummy objective?

This is a **smoke-test trainer**: it validates the fusion architecture, gate behavior, and numeric policy on the data streaming. We then swap in the actual `CondorLoss` / composite loss and targets exactly as in the existing pipeline.

6) How to plug CondorNet into the existing CondorBrain system

The `condor_brain.py` already chooses between CDE and Mamba branches and integrates predicate discovery modules. The predicate discovery and recursive evaluation concepts are already formalized in code.

predicate_discovery

recursive_evaluator

The minimal integration strategy:

1. Keep the existing **predicate selector/combiner** exactly as-is.
2. Produce $p(t)$ = top-K predicate activations (B,T,K).
3. Feed $(x(t), p(t))$ into CondorNetBackbone.
4. Use the same 10 heads and the same loss.

This preserves the “rules separate from model” decision I made, while still letting predicates influence gating/routing.

7) Next steps that will *actually* resurrect Mamba2 value inside CondorNet

If we want the SSM branch’s speed/long memory without NaNs:

1. **Force FP32 inside Mamba** (already in code).
2. Add **input shock guards**:
 - Clip the predicate channels (logits) to a safe range before fusion.
 - Apply robust z-score + clip (we already do for features/targets).

train_condor_brain

3. Add **state growth penalty** (already in code).
4. Add **gating dropout** so the model doesn’t “over-trust” Mamba early.

8) What we can audit that we couldn’t audit before

With CondorNet we can produce a report like:

- “In high IVR + gap risk regime, gate weights shift: $g_{CDE} \uparrow, g_{SSM} \downarrow$.”
- “Feature selection concentrates on [ivr, gap_risk_score, friction_ratio, psar_adaptive] at the moment before spikes.”
- “Predicate bloom signature has high MI with exit_logit 3–5 minutes ahead.”

This is the “institutional gatekeeper” behavior we have discussed.

9) Quick run commands (Lightning / T4)

Smoke test (streaming, stable):

```
python intelligence/train_condor_net.py \  
  --data data/processed/mamba_institutional_2025_1m_v22.csv \  
  --samples 2000 \  
  --batch-size 32 \  
  --epochs 2 \  
  --out models/condor_net_smoke.pth
```

Then we extend to the auditor to treat `condor_net_smoke.pth` as another architecture (adapter reads ckpt keys and instantiates CondorNetBackbone + heads). That's straightforward given CBCI's existing "archaeological inference" pattern.

The *fastest* path to "are we in the ballpark vs being in the Himalayas":

1. Run the smoke trainer above to confirm **no NaNs**, gate behaves, memory stable.
2. Swap the real target/loss heads in (same ordering I already enforce).

`training_monitor.py`

3. Audit CondorNet vs Neural CDE using the streaming auditor (the one that already succeeded on T4).

And yes: the Mamba2 branch is not "over." In CondorNet it becomes a **specialist** that is *used only where it is stable and useful*—exactly the right way to harvest SSD power without letting it destroy the system.